

Tiny Society AI: Research Overview

Research questions, simulation method, and experimental guidance

Tiny Society AI
Multi-Agent Social Simulation and Prediction
17 July 2026

Abstract

Tiny Society AI is an experimental platform for studying small populations of language-model agents. Agents remember events, form relationships, receive different information, and change their views over time. The system records how these individual changes become group patterns. This paper describes the research purpose, simulation design, measurements, reproducibility requirements, and limits of the model. A separate supplement, *Tiny Society AI: Calculations and Measurements*, gives the complete equations and parameter values.

1 Purpose and scope

The project provides a controlled setting for exploring social behavior in LLM-based agent systems. A run begins with a written world, a small population, and an event. The agents then act over 7–30 simulated days. The system tracks memories, relationships, influence, topic stances, and information exposure throughout the run.

The main unit of analysis is the simulated agent. Population measures are built from agent states and actions. They describe the model’s internal society; they are not estimates of real human behavior.

1.1 Research questions

The platform is designed to support questions such as:

- How do memory, reflection, and planning affect later decisions?
- How do relationships and network position affect information exposure?
- When do agent stances converge, and when does disagreement persist?
- Which interactions produce the largest changes in group-level beliefs?
- How do stochastic LLM responses interact with fixed numerical update rules?
- How sensitive are the results to the model provider, starting population, and event?

These are design questions, not completed empirical findings. The repository provides the software and measurements needed to study them.

2 Connection to prior work

Tiny Society AI builds on the architecture introduced by Park et al. in *Generative Agents: Interactive Simulacra of Human Behavior* [1]. Both systems store experiences as natural-language memories, retrieve them using relevance, recency, and importance, form higher-level reflections, and use remembered information during planning and interaction.

This project adapts those ideas for population-level experiments. It adds directed relationships, topic stances, personal information feeds, probabilistic agent selection, fixed background updates, group measurements, and a final forecast summary. Memory relevance uses token overlap rather than embedding similarity, and recall includes a penalty for returning near-duplicate memories.

The cited work is the starting point for the agent architecture. The formulas and thresholds in this project are implementation choices and are documented separately in *Calculations and Measurements*.

3 Model definition

3.1 Agent state

Each agent contains the following state:

State	Role in the simulation
Traits and goals	Define the character context used for planning and action generation.
Memories	Store past observations, actions, advice, and reflections as text.
Relationships	Record directed social categories, strength, and interaction history.
Influence	Represents visibility or standing within the simulated population.
Topic stances	Store an opinion from -1 to 1 for each extracted topic.
Personal feed	Contains the subset of actions the agent witnessed, ranked for relevance.
Groups	Connect agents through shared organizations or social membership.

3.2 Initialization

A run is defined by a world prompt, initial characters, a starting event, and an optional user prediction. Characters can be written by the user or generated to reach a target population. The engine extracts a world graph and a set of stance topics before the first day. Initial relationships and influence values become the baseline for later measurements.

3.3 Daily update cycle

Each simulated day follows seven steps:

1. **Select active agents.** A weighted random sampler uses event relevance, influence, charged relationships, and group membership. The default limit is eight agents per day.
2. **Build individual context.** Each selected agent receives its traits, goals, relationships, plan, relevant memories, and personal feed.
3. **Generate structured actions.** The LLM returns an action, targets, emotional response, relationship intent, new memory, explanation, and any stance shift.
4. **Interpret social effects.** A recipient can interpret an action through its own personality and history. Charged relationships can produce short follow-up exchanges.
5. **Apply state changes.** The engine validates and applies relationship, influence, memory, and stance updates in a fixed order.
6. **Update background agents.** Agents without an LLM turn still receive fixed daily bond, group, stance, and influence updates.
7. **Measure the population.** The engine records social structure, influence changes, topic means, disagreement, and forecast confidence.

A day contains morning, afternoon, evening, and night phases. LLM calls for selected agents can run concurrently. Their returned actions are applied sequentially so the state-update order is clear and repeatable for a fixed set of generated outputs.

4 Measurements and interpretation

The system produces daily measurements in four groups.

Group	Recorded outputs
Social structure	Relationship counts, average bond strength, isolated-agent fraction, group connections, and relationship turning points.
Influence	Agents with the largest gains or losses relative to the start of the run.
Beliefs	Mean stance and population disagreement for every represented topic.
Forecast summary	Consensus score, pivotal days, final topic values, narrative summary, and an optional assessment of the user's prediction.

Forecast confidence is a consensus score. A high value means that the simulated agents hold similar stances. It does not mean the forecast is likely to be correct. Relationship categories, influence, and centrality are also model-specific values rather than direct measurements of human society.

Pivotal days are the three days with the largest total movement across topic means. They help a reader locate major changes in a run, but they do not prove that a single action caused the final outcome.

The companion paper *Tiny Society AI: Calculations and Measurements* defines every formula, range, threshold, and update rule used by these measures.

5 Experimental protocol

5.1 Run specification

Every reported run should include enough information to reproduce its setup:

- world prompt, starting event, and initial character definitions;
- LLM provider and exact model identifiers;
- simulation length and daily reasoning-agent limit;
- concurrency, model-tier, and recency settings;
- any user advice, injected event, or character added during the run; and
- the software revision or commit identifier.

Saved world data should be retained when possible. It contains the initial state and the daily trajectory needed to inspect the result.

5.2 Comparison design

For a controlled comparison, change one factor at a time while keeping the remaining setup fixed. Examples include changing the LLM model, turning user intervention on or off, or varying the number of active agents. Real LLMs can return different outputs for the same input, so a comparison should use repeated runs and report the distribution of results rather than a single example.

The mock provider returns deterministic structured data and is useful for software tests. It does not model real language behavior and should not be used to support claims about agent quality.

5.3 Suggested outcome reporting

At minimum, a report should include final topic means, topic disagreement, relationship counts, influence changes, pivotal days, and the number of completed agent actions. Charts should show the daily trajectory, not only the final value. Unexpected or failed runs should be counted and reported.

6 Software architecture

The research interface is a Next.js application. It communicates with a FastAPI backend through HTTP and Server-Sent Events. The backend contains the simulation loop, LLM adapters, state store, measurements, and optional Supabase persistence.

Component	Responsibility
World graph	Extract entities, relationships, groups, power structure, and topics.
Selector	Choose the agents that receive an LLM reasoning turn.
Reasoner	Build context and parse a structured action.
Memory and reflection	Retrieve past experiences and synthesize higher-level observations.
Observation	Route witnessed actions and rank personal feeds.
Perception	Adjust social effects using recipient history and personality.
Applicator	Validate and apply state changes.
Deterministic layer	Update agents without an LLM turn.
Metrics and reporter	Aggregate daily state and produce the final report.

The engine supports Anthropic, OpenAI-compatible, and mock providers. Routine and player-facing calls can use different model tiers. Concurrent LLM generation reduces run time, while sequential state mutation keeps the order of effects explicit.

7 Limitations and responsible use

- The model has not been validated against longitudinal human interaction data.
- Results can change with provider, model version, prompt wording, and sampling behavior.
- Numeric weights and thresholds are design choices, not estimated human parameters.
- The intended scale is tens of agents, not population-scale social forecasting.
- A coherent narrative is not evidence that the simulated mechanism is realistic.
- User interventions make a run path-dependent and must be included in any report.
- The mock provider verifies software behavior but not real-LLM output quality.

Agents based on real people require additional care. Researchers should avoid private or sensitive personal data, obtain appropriate consent, document the source of character information, and avoid presenting simulated statements as statements

made by the real person. Institutional or legal review may be needed depending on the study and jurisdiction.

8 Reproducibility checklist

Before publishing or comparing results, confirm that:

- the initial world and agent data are archived;
- provider and model versions are recorded;
- configuration values and interventions are listed;
- repeated runs are used for stochastic providers;
- failures and excluded runs are reported;
- raw daily measurements are retained; and
- claims are limited to what the simulation actually measures.

References

- [1] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, 2023, pp. 1–22. doi: 10.1145/3586183.3606763.